

XyzyBlog 博客系统 — 项目答辩讲稿

适用场景：课堂项目验收 / 答辩演示
建议用时：8-12 分钟
使用方法：用 Typora 或 VS Code 打开此文件，导出为 PDF 即可

第 1 页：封面

XyzyBlog 个人博客系统
—— 后端项目答辩

项目类型：Spring Boot 综合实训
技术栈：Spring Boot 3 + MyBatis-Plus
 Spring Security + JWT + Redis

答辩人：_____
日期：2026 年 __ 月 __ 日

第 2 页：项目定位 (1 分钟)

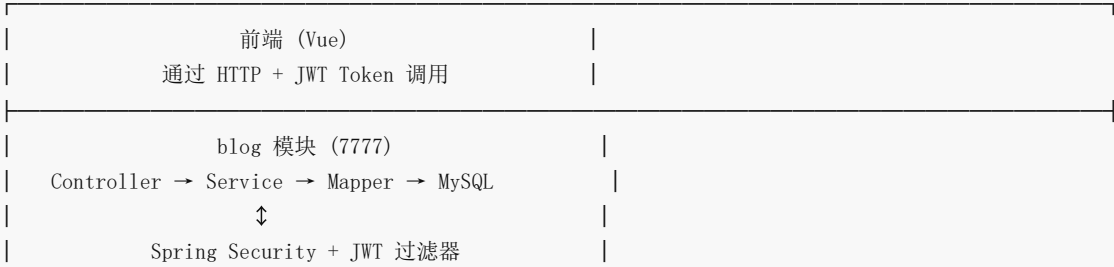
这是一句话介绍：

用 Spring Boot 搭建的博客系统后端，提供文章、分类、评论、友链、用户注册登录、浏览量统计等功能，前后端分离架构，前端通过 API 调用后端接口。

项目规模：

指标	数据
代码文件	100+ 个 Java 文件
数据表	11 张
API 接口	前台 14 个 + 后台 30+ 个
模块数	3 个 Maven 子模块
涉及技术	10+ 种

第 3 页：技术架构 (1-2 分钟)



Redis (登录会话 + 浏览量)	
AOP 日志切面	
定时任务 (Redis → MySQL)	
framework 公共模块	
Entity / VO / DTO / Mapper / Utils / Handler	
admin 模块 (8989)	
管理员 CRUD + RBAC 权限控制	

技术选型理由：

技术	为什么用
Spring Boot 3	主流框架，自动配置，约定优于配置
MyBatis-Plus	零 SQL 实现 CRUD，分页插件一行搞定
Spring Security	成熟的认证授权框架，和 Spring Boot 无缝集成
JWT	无状态 Token，适合前后端分离
Redis	高性能缓存，存登录会话和浏览量
AOP	非侵入式记录接口日志，不污染业务代码
FastJson	阿里出品的 JSON 库，序列化快

🚀 第 4 页：项目结构 (1 分钟)

XyzyBlog/	
├── framework/	公共模块
├── domain/entity	12 个实体类 (对应 11 张表)
├── domain/vo	10 个 VO (返回给前端)
├── domain/dto	8 个 DTO (接收前端参数)
├── mapper/	11 个 Mapper 接口
├── service/	8 个 Service + 实现类
├── handler/	异常处理器 + Security 处理器
├── utils/	6 个工具类
└── config/	Redis 配置
├── blog/	前台模块 (端口 7777)
├── controller/	6 个 Controller
├── config/	Security + MyBatisPlus 配置
├── filter/	JWT 过滤器
├── aspect/	AOP 日志切面
├── runner/	浏览量初始化
└── job/	浏览量定时同步
├── admin/	后台模块 (端口 8989)
├── controller/	7 个管理 Controller

└── config/	后台 Security 配置
└── sql/init.sql	数据库初始化脚本

✦ 第 5 页：核心功能演示路线（2-3 分钟）

按这个顺序演示，由浅入深：

第一步：无需登录的查询接口（展示基础 CRUD）

- ① GET /article/hotArticleList → 热门文章 Top 10
- ② GET /category/getCategoryList → 分类下拉列表
- ③ GET /link/getAllLink → 友情链接
- ④ GET /article/articleList?pageNum=1&pageSize=10 → 文章分页
- ⑤ GET /article/1 → 文章详情（含正文）

讲解要点：这些都是 GET 请求，浏览器直接能访问。实现上全部是 Controller → Service → Mapper 三层调用，MyBatis-Plus 的 LambdaQueryWrapper 构建条件，Page 对象分页，BeanCopyUtils 把 Entity 转 VO 返回。

第二步：用户注册和登录

- ⑥ POST /user/register → 注册（6 项校验）
- ⑦ POST /login → 登录（返回 JWT Token）

讲解要点：

- 注册做了 6 项校验：用户名/密码/昵称/邮箱不能为空 + 用户名/昵称/邮箱不能重复
- 密码用 BCryptPasswordEncoder 加密存储，数据库中不存明文
- 登录流程：AuthenticationManager 认证 → UserDetailsServiceImpl 查库验证 → 生成 JWT → 用户信息存 Redis → 返回 Token

第三步：需要登录的接口（展示认证机制）

- ⑧ POST /logout → 退出（清除 Redis 会话）
- ⑨ GET /user/userInfo → 查个人信息（从 Token 中识别用户）
- ⑩ PUT /user/userInfo → 改个人信息
- ⑪ POST /comment → 发表评论

讲解要点：

- 前端请求头带上 token，后端 JwtAuthenticationTokenFilter 拦截
- Filter 做的事：取 Header 中的 token → 解析 JWT 拿 userId → 去 Redis 查 bloglogin:userId → 有就放入 SecurityContext 标记已登录
- 没带 Token 访问 /user/userInfo → 返回 {code: 401, msg: “需要登录后操作”}

第四步：评论的树形结构

- ⑫ GET /comment/commentList?articleId=1

讲解要点：

- 评论分两级：一级评论（rootId = -1）和子回复（rootId = 父评论 id）

- 后端返回的数据里 `CommentVo` 有 `children` 字段，一级评论嵌着子评论列表
- 前端直接 `v-for` 嵌套渲染即可

🔥 第 6 页：技术亮点详解（2-3 分钟）

亮点一：JWT + Redis 双重认证

登录 → JwtUtil 生成 Token（24h 有效，HS256 签名）
→ Redis 存 LoginUser（key = bloglogin:userId）

每次请求 → Filter 拦截
→ 解析 JWT 拿 userId
→ Redis 验证会话是否存在
→ 存在 → 标记已登录
→ 不存在 → 401 “需要登录后操作”

优势：退出登录只需删 Redis，服务端主动可控。Token 本身无状态，适合分布式。

亮点二：Redis 浏览量 + 定时任务

启动时：ViewCountRunner 把 MySQL 浏览量加载到 Redis Hash
article:viewCount → {"1": 1520, "2": 890, "3": 2300}

用户阅读：Redis HINCRBY article:viewCount 1 1
（不直接写 MySQL，避免频繁磁盘 IO）

每 5 分钟：UpdateViewCountJob 把 Redis 数据批量同步回 MySQL
UPDATE t_article SET view_count=? WHERE id=?

优势：高并发下不卡数据库，Redis 的 Hash 自增是原子操作。

亮点三：AOP 日志

```
@SystemLog(businessName = "热门文章排行")    // 只需要加一行注解
@GetMapping("/hotArticleList")
public ResponseResult hotArticleList() { ... }
```

控制台自动输出：

```
URL: http://localhost:7777/article/hotArticleList
BusinessName: 热门文章排行
HTTP Method: GET
IP: 127.0.0.1
Response: {"code":200, ...}
```

优势：非侵入式，不修改业务代码，注解加在哪就记录到哪。

亮点四：RBAC 五表权限模型

```
t_user ——> t_user_role ——> t_role ——> t_role_menu ——> t_menu
                                     |
                                     perms 权限标识
```

登录时查用户所有权限字符串（如 `content:article:add`），存入 `LoginUser`，`@PreAuthorize` 注解即可控制接口权限。

亮点五：统一异常处理

```
@RestControllerAdvice // 全局拦截
public class GlobalExceptionHandler {
    // 不管哪里抛异常，前端永远拿到统一格式：
    // {"code": 505, "msg": "用户名或密码错误", "data": null}
}
```

🚀 第 7 页：数据库设计（1 分钟）

11 张表，分三组：

业务表（前台用）：

```
t_article    文章 (title, content, summary, category_id, view_count, is_top)
t_category   分类 (name, pid 支持父子分类)
t_comment    评论 (type, root_id 支持嵌套回复)
t_link       友链 (name, logo, address, status 审核状态)
t_tag        标签 (name)
t_article_tag 文章-标签多对多关联
```

权限表（RBAC）：

```
t_user       用户
t_role       角色
t_menu       菜单/权限
t_user_role  用户-角色关联
t_role_menu  角色-菜单关联
```

通用设计：每张表都有 `del_flag`（逻辑删除）、`create_time/create_by/update_time/update_by`（审计字段，MyBatis-Plus 自动填充）。

🚀 第 8 页：项目亮点总结（30 秒）

说这 5 个词就够了：

1. **三层架构** — *Controller/Service/Mapper 职责清晰*
2. **统一返回** — 所有接口返回 `{code, msg, data}` 格式
3. **JWT 认证** — 无状态 `Token` + `Redis` 会话管理
4. **读写分离** — 浏览量先写 `Redis`，定时同步 `MySQL`
5. **多模块** — `Maven` 分 `framework/blog/admin`，公共代码复用

🚀 第 9 页：验收演示 Checklist

验收时逐项打勾：

环境准备：

- ☐ MySQL 已启动，执行了 `sql/init.sql`
- ☐ Redis 已启动（或 Docker： `docker compose up -d`）
- ☐ IDEA 运行 `BlogApplication`，端口 `7777` 无报错

无需登录的接口（浏览器直接访问）：

- ☐ `http://localhost:7777/article/hotArticleList` → 返回 3 篇文章 JSON
- ☐ `http://localhost:7777/category/getCategoryList` → 返回 3 个分类
- ☐ `http://localhost:7777/link/getAllLink` → 返回 2 个友链
- ☐ `http://localhost:7777/article/articleList?pageNum=1&pageSize=10` → 分页数据
- ☐ `http://localhost:7777/article/1` → 文章正文
- ☐ `http://localhost:7777/comment/commentList?articleId=1` → 评论树

需要登录的接口（Postman 演示）：

- ☐ `POST /user/register` — 注册新用户，校验失败返回对应错误码
- ☐ `POST /login` — 用 `admin/admin123` 登录，拿到 token
- ☐ `GET /user/userInfo` — 不带 token → 401；带 token → 返回用户信息
- ☐ `POST /comment` — 带 token 发表评论
- ☐ `POST /logout` — 退出后 Redis 中对应 key 被删除

技术加分项：

- ☐ 控制台看 AOP 日志输出（URL、IP、参数、返回值）
- ☐ 用 `redis-cli` 看 `HGETALL article:viewCount` 验证浏览量
- ☐ 等 5 分钟后看 MySQL 浏览量是否更新

🔴 可能被问到的问题

问题	参考答案
JWT 过期了怎么办？	前端收到 401 后跳转登录页，重新登录获取新 Token。Token 有效期 24 小时
Redis 挂了影响什么？	登录状态丢失（需重新登录），浏览量计数暂时失效。MySQL 数据不会丢
为什么浏览量不直接写 MySQL？	高并发下频繁 UPDATE 会锁表。Redis 单线程原子操作，批量同步更高效
密码怎么存的？	BCrypt 加密，同一次加密每次结果不同（加盐），无法反推原文
怎么区分普通用户和管理员？	<code>t_user.type</code> 字段， <code>'0'</code> 普通用户， <code>'1'</code> 管理员。后台模块额外校验角色
如何加一个新接口？	三步：① Controller 写路由 ② Service 写逻辑 ③ 如果新表就加 Entity + Mapper
前后端怎么联调？	前端项目 <code>npm run dev</code> 启动，Vue 配置代理把 <code>/api</code> 转发到 <code>localhost:7777</code>
项目能部署到服务器吗？	可以。 <code>mvn package</code> 打 jar 包， <code>java -jar blog.jar</code> 直接运行，内嵌 Tomcat

附录：快速启动命令

```
# 1. 初始化数据库（在 MySQL 中执行）
source sql/init.sql;

# 2. 启动 Redis（如果没有就用 Docker）
docker compose -f deploy/docker-compose.yml up -d

# 3. 编译项目
mvn -pl blog -am clean package -DskipTests

# 4. 启动博客前台
java -jar blog/target/blog-1.0-SNAPSHOT.jar

# 5. 浏览器测试
open http://localhost:7777/article/hotArticleList
```